



UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Departamento Académico de Ingeniería Informática y de Sistemas

COMPUTACIÓN GRÁFICA I

Práctica N.º 12

GENERACIÓN DE CURVAS PARAMÉTRICAS EN 3D

Evaluación de curvas de Bézier cúbicas en el Vertex Shader

Estudiante: Efrain Vitorino Marin

Código: 160337

Profesor: Dr. Hans Harley Ccacyahuillca Bejar

Repositorio: github.com/lovito99/labografica/tree/main/lab012

Resumen

Se implementó la curva cúbica del código base y una curva compuesta de dos segmentos, evaluadas íntegramente por el *vertex shader*. La CPU transfiere los puntos de control como *uniforms* y duplica el dominio discreto $t \in [0, 1]$ para ambos tramos. La unión satisface continuidad geométrica y diferencial C^1 . Además, los extremos A_0 y B_3 pueden modificarse en los tres ejes mediante el teclado.

1. Objetivos

- Evaluar en paralelo los polinomios de Bernstein en la GPU.
- Construir dos segmentos Bézier cúbicos con continuidad C^1 estricta.
- Actualizar puntos de control mediante variables uniformes.
- implementar interacción tridimensional independiente del framerate.

2. Fundamento matemático del pipeline

Para cuatro puntos de control, la curva cúbica es

$$C(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3, \quad 0 \leq t \leq 1.$$

La CPU no calcula posiciones de la curva. El VBO contiene solamente 241 escalares equidistantes $t_i = i/240$. El mismo VAO se dibuja dos veces: primero con los *uniforms* $A_0, \dots, A_3$ y luego con $B_0, \dots, B_3$. Esta “duplicación” es modular: se reutiliza el dominio, pero cada llamada `glDrawArrays` recibe otro conjunto de puntos de control. Cada invocación del *vertex shader* evalúa en paralelo un valor de t .

Listing 1: Evaluación de Bernstein en el vertex shader.

```

1 float t = aT, u = 1.0 - t;
2 vec3 pos = (u*u*u)*P0 + (3.0*u*u*t)*P1
3           + (3.0*u*t*t)*P2 + (t*t*t)*P3;
4 gl_Position = projection * view * model * vec4(pos, 1.0);

```

2.1. Ejemplo base implementado

Se conservaron los tres componentes solicitados en el PDF: `curve.vert`, `curve.frag` y `mainCurves.cpp`. El ejemplo base emplea 200 subdivisiones, es decir, 201 muestras incluyendo los extremos.

Listing 2: Inicialización semántica del atributo escalar t .

```

1 constexpr int segments = 200;
2 std::vector<float> values(segments + 1);

```

```
3 for (int i = 0; i <= segments; ++i)
4     values[i] = static_cast<float>(i) / segments;
5
6 glBufferData(GL_ARRAY_BUFFER,
7             values.size() * sizeof(float),
8             values.data(), GL_STATIC_DRAW);
9 glVertexAttribPointer(0, 1, GL_FLOAT, GL_FALSE,
10                      sizeof(float), nullptr);
```

Listing 3: Puntos de control del ejemplo base del PDF.

```
1 const glm::vec3 p0(-10, -5, 0);
2 const glm::vec3 p1(-5, 10, 5);
3 const glm::vec3 p2(5, -10, -5);
4 const glm::vec3 p3(10, 5, 0);
```



Figura 1: Ejemplo base: curva Bézier cúbica evaluada en el vertex shader; extremo inicial cian y extremo final magenta.

3. Continuidad estricta en la unión

La continuidad C^0 exige que ambos segmentos compartan el punto de unión:

$$B_0 = A_3.$$

Las derivadas de una Bézier cúbica en sus extremos son

$$C'_A(1) = 3(A_3 - A_2), \quad C'_B(0) = 3(B_1 - B_0).$$

Al imponer

$$B_1 = A_3 + (A_3 - A_2)$$

se obtiene $B_1 - B_0 = A_3 - A_2$ y, por tanto, $C'_A(1) = C'_B(0)$. Coinciden posición, dirección y magnitud de la tangente; la unión es C^1 , no solo visualmente suave.

Listing 4: Restricción recalculada en cada cuadro.

```
1 const glm::vec3 b0 = a3;
2 const glm::vec3 b1 = a3 + (a3 - a2);
```

4. Diagrama algorítmico

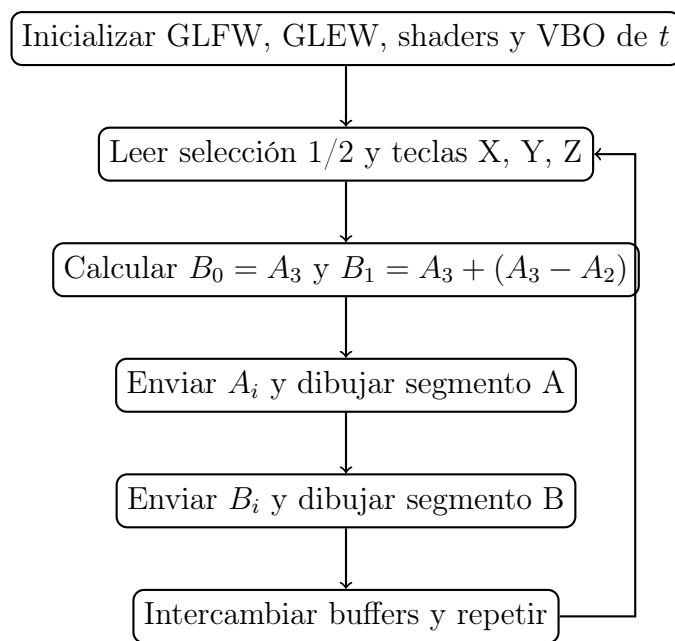


Figura 2: Flujo propio de la aplicación interactiva.

5. Control dinámico 3D

Las teclas 1 y 2 seleccionan A_0 o B_3 . Las flechas alteran x e y , mientras W/S alteran z . El desplazamiento se multiplica por el tiempo entre cuadros para mantener una velocidad uniforme. Como los puntos se envían en cada iteración, la deformación aparece inmediatamente sin reconstruir el VBO.

6. Buenas prácticas

La creación de ventana, compilación, generación del dominio y dibujo están modularizados en `bezierComun.hpp`. Se comprueba la compilación y enlace de shaders, se eliminan VAO, VBO, programa y ventana, y se usa tamaño semántico `sizeof(float)`. El VBO es inmutable porque la interacción solo modifica *uniforms*.

7. Resultados y verificación

Se verificó mediante compilación y ejecución que cada programa funciona con C++17 y OpenGL 3.3. La curva base presenta el degradado cian-magenta y rotación global. En la actividad, los dos tramos conservan un color común en la unión y no muestran quiebre al mover los extremos.

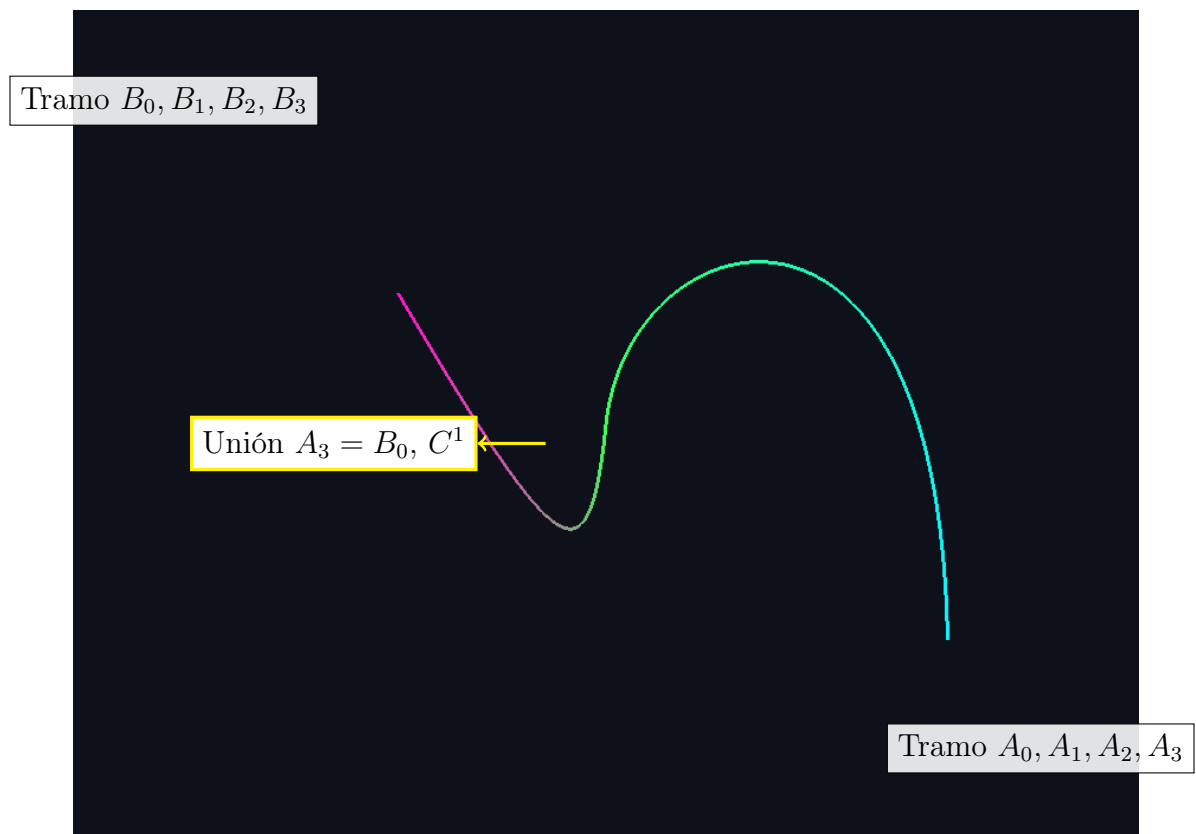


Figura 3: Resultado ejecutado y etiquetado de la curva Bézier compuesta.

8. Conclusiones

- Enviar únicamente t reduce el trabajo geométrico de la CPU y aprovecha la evaluación paralela del pipeline.
- Las ecuaciones de los puntos de control prueban formalmente la continuidad C^1 en la unión.

- Los *uniforms* permiten deformación 3D en tiempo real sin transferir un nuevo conjunto de vértices.